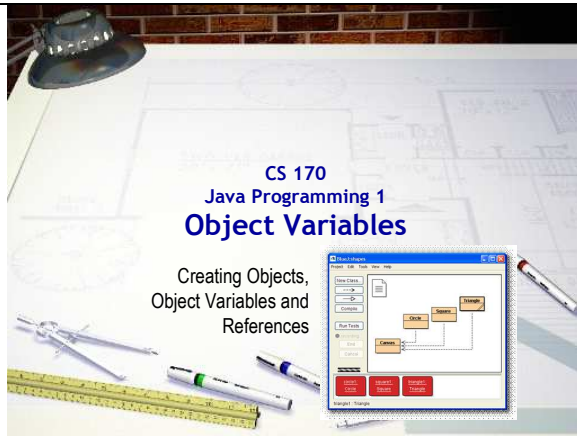




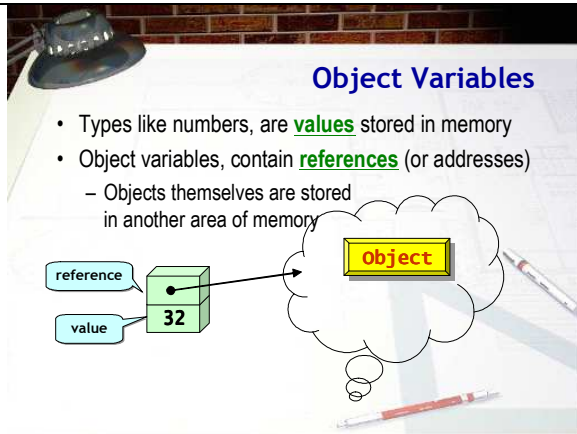
Slide 1
CS 170
Java Programming 1
Object Variables
 Duration: 00:00:52
 Advance mode: Auto

Notes:

Hi Folks. Welcome to the CS 170, Java Programming 1 lecture on Object Variables.

This past week in class you were introduced to BlueJ IDE and the Shapes project, where you interactively created Circle, Square and Triangle objects and placed them on the BlueJ object bench like this. You also used BlueJ's Code Pad facility to create simple numeric variables, assign values, and display results using Java code, rather than the BlueJ menus.

In the online lessons this week, we'll pick up where the in-class lecture left off, learning to write Java code that creates and uses different kinds of objects. In this lecture we'll tackle the topic of writing code to create object variables, create objects and store object references in object variables.



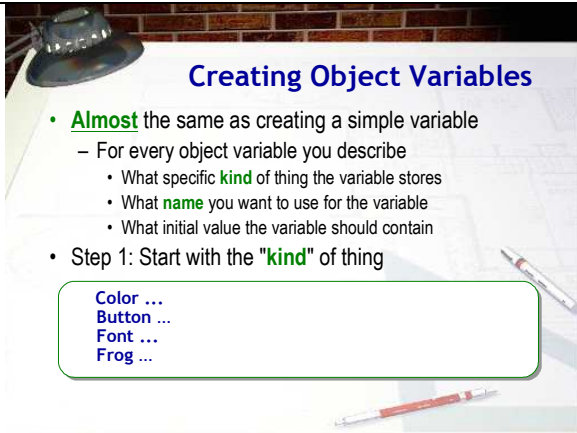
Slide 2
Object Variables
 Duration: 00:01:16
 Advance mode: Auto

Notes:

Java really has two major ways of representing data: primitive types like numbers and characters, and object types like Circles, Rectangles and Strings. We'll look at this difference in a little greater depth later this week. For right now, though, I want to introduce you to a picture that might make this a little clearer.

We call primitive types, like numbers, values, and when you create a primitive variable, the actual value fits inside the variable, like a bucket or shoebox. In the picture here, the green box represents a variable which contains the value 32.

Object variables, on the other hand, do not contain values; instead, they contain references. A reference is just another name for a memory address. When you create an object variable, rather than storing a value in a memory, you create an object (in a different area of memory) and store its address inside the variable. Thus, when you work with objects, you always have two pieces: the object variable (which contains a reference), and the object itself. When you work with values, there is only one piece: the variable which contains a value.



Creating Object Variables

- **Almost** the same as creating a simple variable
 - For every object variable you describe
 - What specific **kind** of thing the variable stores
 - What **name** you want to use for the variable
 - What initial value the variable should contain
- Step 1: Start with the "**kind**" of thing

Color ...
 Button ...
 Font ...
 Frog ...

Slide 3 

Creating Object Variables

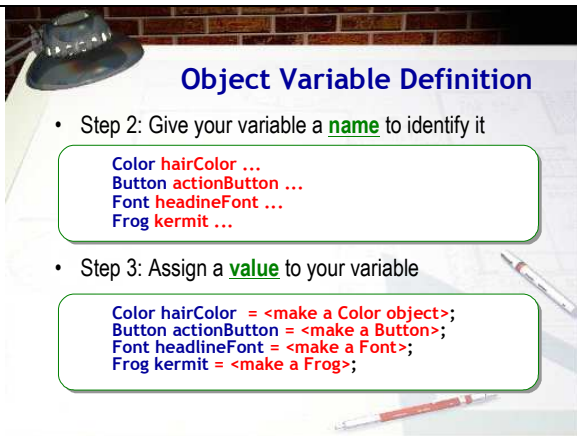
Duration: 00:00:48
Advance mode: Auto

Notes:

Creating an object variable is almost exactly the same as creating a primitive variable. With both kinds of variables, you have to explicitly specify three things:

- What kind (type) of thing will be stored in the variable? This, recall, is the data type of the variable
- What name you want to use for the variable. We talked about the different syntax rules for naming variables in class this week.
- What initial value the variable should have.

Let's see how this works with objects. Remember that step 1 is to specify the "kind" of thing our variable will store. Here are some examples: Color, Button, Font, Frog



Object Variable Definition

- Step 2: Give your variable a **name** to identify it

Color **hairColor** ...
 Button **actionButton** ...
 Font **headlineFont** ...
 Frog **kermit** ...

- Step 3: Assign a **value** to your variable

Color **hairColor** = <make a Color object>;
 Button **actionButton** = <make a Button>;
 Font **headlineFont** = <make a Font>;
 Frog **kermit** = <make a Frog>;

Slide 4 

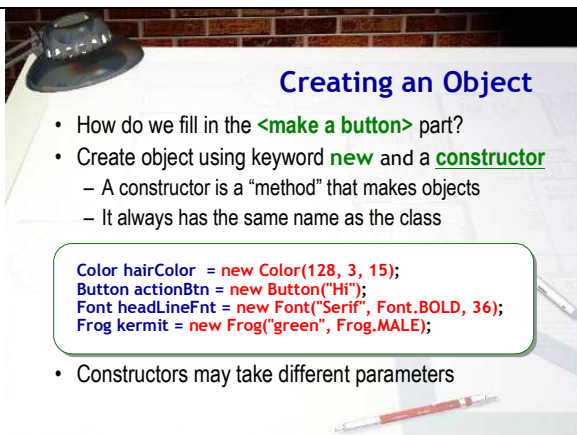
Object Variable Definition

Duration: 00:01:10
Advance mode: Auto

Notes:

Step 2 is to give your variable a name to identify it. Here are some examples. hairColor, actionButton, headlineFont, kermit. Notice that the convention in Java is to start variables with lowercase and to use embedded capitals to separate words. These conventions aren't syntax rules, but they'll help other Java programmers to distinguish between identifiers that refer to variables and identifiers that refer to classes when reading your code.

Step 3 is to provide an initial value for your variable. You do that using the assignment operator, like this. At this point, we're kind of in a quandary. How do we create an object? What do we put on the right-hand-side of the assignment operator? If our variable was a number, for instance, we'd expect to write something like int someNumber = 27; Since these variables are all objects, however, we need to learn how to construct an object; we can't just use a literal value like 27. Fortunately, constructing an object is easy.



Creating an Object

- How do we fill in the <make a button> part?
- Create object using keyword **new** and a **constructor**
 - A constructor is a "method" that makes objects
 - It always has the same name as the class

Color **hairColor** = new Color(128, 3, 15);
 Button **actionBtn** = new Button("Hi");
 Font **headLineFnt** = new Font("Serif", Font.BOLD, 36);
 Frog **kermit** = new Frog("green", Frog.MALE);

- Constructors may take different parameters

Slide 5 

Creating an Object

Notes:

How to we fill in the <make a button> part? That's easy!

To construct an object—any object—you just use a constructor method, along with the keyword new. A constructor is simply a specialized kind of method that initializes objects. If a class represents a blueprint that describes the attributes and methods of an object, then the constructor is the factory that creates the goods. To make things even easier, constructors always have the same name as the thing you're trying to create. Since we're trying to create a Color object, for instance, we know that the constructor is a method named Color(). Now we can finish what we started.

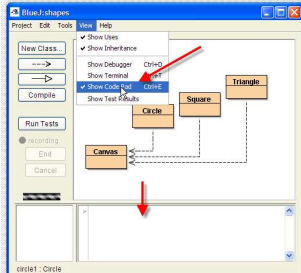
As you can see here, different constructors can take different

Duration: 00:01:11
Advance mode: Auto

numbers and types of parameters. That's because each class needs very specific "raw materials", so to speak, to create its own objects. We'll talk more about parameters in the next lecture. For right now, though, I want you to experiment with writing some code that creates object variables and the code that creates the objects themselves, making sure you understand the difference.

Express Yourself

- Exercise 1:** Make sure the Shapes project is open in BlueJ, and the CodePad window is displayed. Shoot me a screen-shot.



The screenshot shows the BlueJ IDE with the 'Shapes' project open. The 'New Class' dialog is open, and the 'Shapes' package is selected. The 'Code Pad' window is also visible, showing the 'Circle' class. A red arrow points to the 'Code Pad' window, and another red arrow points to the 'Shapes' package in the 'New Class' dialog.

Slide 6

Express Yourself

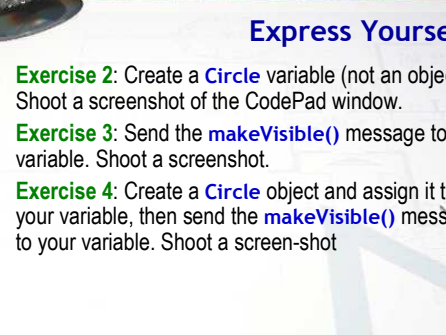
Duration: 00:00:38

Advance mode: Auto

Notes:

To get started, open the Shapes project in BlueJ, and open the Code Pad. Then, start your "in-class" lab exercise document. In each of the mini-lectures this week, I'll number the exercises starting at 1. When you create your IC document, please start a new section for each online lesson so that I can tell the similarly numbered questions apart.

For Exercise 1 in this section, just shoot me a screen-shot of the IDE similar to the one shown here.



Express Yourself

- **Exercise 2:** Create a **Circle** variable (not an object). Shoot a screenshot of the CodePad window.
- **Exercise 3:** Send the **makeVisible()** message to your variable. Shoot a screenshot.
- **Exercise 4:** Create a **Circle** object and assign it to your variable, then send the **makeVisible()** message to your variable. Shoot a screen-shot

Slide 7

Express Yourself

Duration: 00:01:24

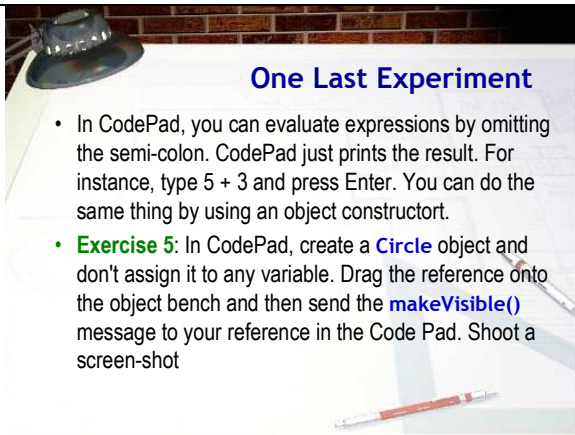
Advance mode: Auto

Notes:

For Exercise 2, create a Circle variable named c1. Since this is a statement, you'll need to end it with a semicolon. Note that we don't want to create a Circle object, just the variable, so you won't use the assignment operator, the new operator or a constructor. Shoot me a screen-shot of the variable in the Code Pad window.

For Exercise 3, send the `makeVisible()` message to your new variable. You should get an error message when you try to do this, because even though you have a `Circle` variable, `c1`, the variable doesn't yet contain a reference to a `Circle` object. Shoot me a screenshot of the error message that appears.

For Exercise 4, create a Circle object and assign it to your new variable. Once you've done that, then send the `makeVisible()` message to your variable again. This time it should work and you should see the screen with the Circle pop up. If you have difficulty with the assignment, remember that you don't need to create a new variable; you already have the variable `c1` which you created in Exercise 2. Shoot me two screen-shots for this exercise: one of the Code Pad window and one showing the circle as it appears.



One Last Experiment

- In CodePad, you can evaluate expressions by omitting the semi-colon. CodePad just prints the result. For instance, type `5 + 3` and press Enter. You can do the same thing by using an object constructor.
- **Exercise 5:** In CodePad, create a `Circle` object and don't assign it to any variable. Drag the reference onto the object bench and then send the `makeVisible()` message to your reference in the Code Pad. Shoot a screen-shot

Slide 8

One Last Experiment

Duration: 00:01:27

Advance mode: Auto

Notes:

Let's try one final experiment. Do you remember in class when you typed an expression, like `5 + 3` into Code Pad. When you leave off the semicolon, this is called an expression, and Code Pad prints both the type (`int`) and the value `8`. We can do the same thing with objects, by simply using `new` and an object constructor. Let's try it out.

In Code Pad, create a `Circle` object using `new` and the `Circle` constructor. Do not create a new object variable and don't assign the reference to an existing variable. When you create the object, don't end the line with a semicolon; just hit Enter.

Note that instead of printing out the value of the `Circle` object, BlueJ creates a little red Object Reference icon. Drag that object reference over to the Object Bench. Notice when you drop it, you're given a chance to name it. Give the object reference a name, and then, in Code Pad, send the reference the `makeVisible()` message. Note that the `Circle` appears on the screen just like when you sent the message to the variables you created in Code Pad.

In fact, objects that appear in the Object Bench, whether you create them using Code Pad or interactively by right-clicking one of the class icons, aren't really objects at all, but are object variables that you can manipulate by sending messages.